Ask and remember

First, ask how many coins the player has. Store their answer.

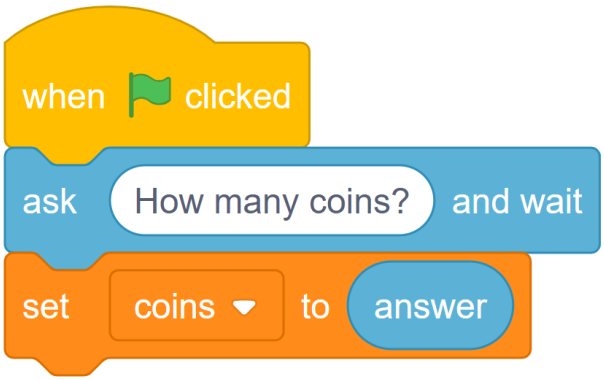
Name: _____ Class: _____ Date: _____

1. Open **scratch.mit.edu** and choose **Create**. Keep the cat selected and open the **Code** tab. These pictures use English block names.

2. Click **Variables** (orange), then **Make a Variable**. Type **coins**, choose **For all sprites**, and click **OK**. Tick the box beside coins so its value is visible on the Stage.

Find the blocks: Events (yellow) - Sensing (light blue) - Variables (orange).

3. Drag these three blocks into the Code area. Join them in the order shown. Type the question into the white box in **ask**.



How to fill the last block

In **set [coins] to (0)**, choose **coins** from the small arrow menu.

From **Sensing**, drag the oval **answer** block into the white **0**. Do not type the word answer.

4. Click the green flag. Type **9** in the answer box and press **Enter**. Click the flag again and try **10**.

Type	coins should show	What did yours show?
9	9	
10	10	

Check: All three blocks must touch. The orange set block must contain the light-blue oval answer block.

Save: File > Save to your computer. Use **Y7_Week5_Gate_YourName.sb3**.

Make the coin gate

The gate opens when the player has at least 10 coins.

Use whole-number coin counts: 0, 1, 2, 3, ... For these inputs, **coins > 9** means 10 coins or more.

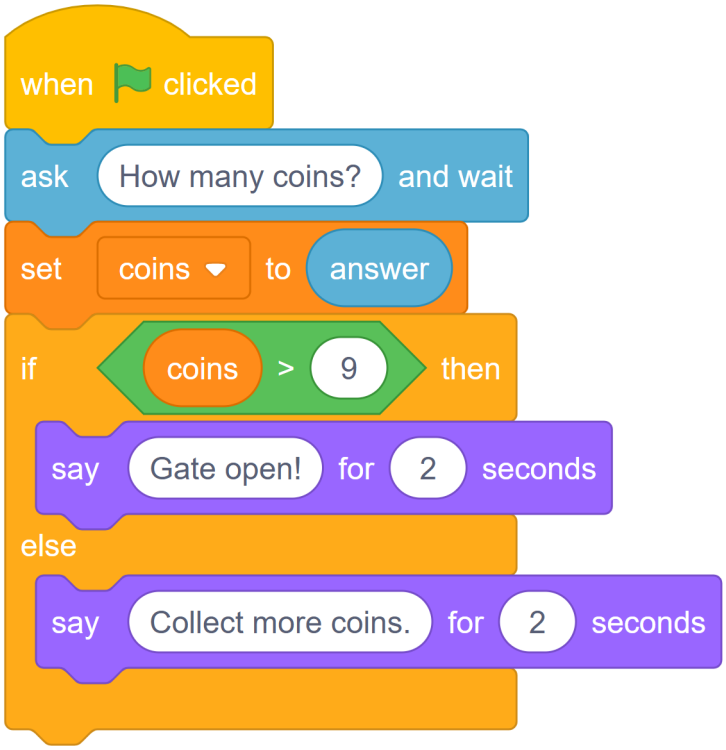
Find the blocks: Operators (green) - Control (orange) - Looks (purple).

1. In **Operators**, drag out **>**. Drag the orange oval **coins** into its left input. Type **9** in the right input.



2. From **Control**, use **if ... then ... else** (the block with two spaces inside). Attach it below set coins. Put the green comparison into its hexagonal hole.

3. From **Looks**, add a **say ... for 2 seconds** block inside each space. Type the messages exactly as shown.



Check the positions

Above else:
Gate open!

Below else:
Collect more coins.

Both purple blocks go **inside** the orange block.

Try it: 9 must say Collect more coins. 10 must say Gate open! If the result is wrong, check that the test is coins > 9, not 9 > coins.

Test before you change it

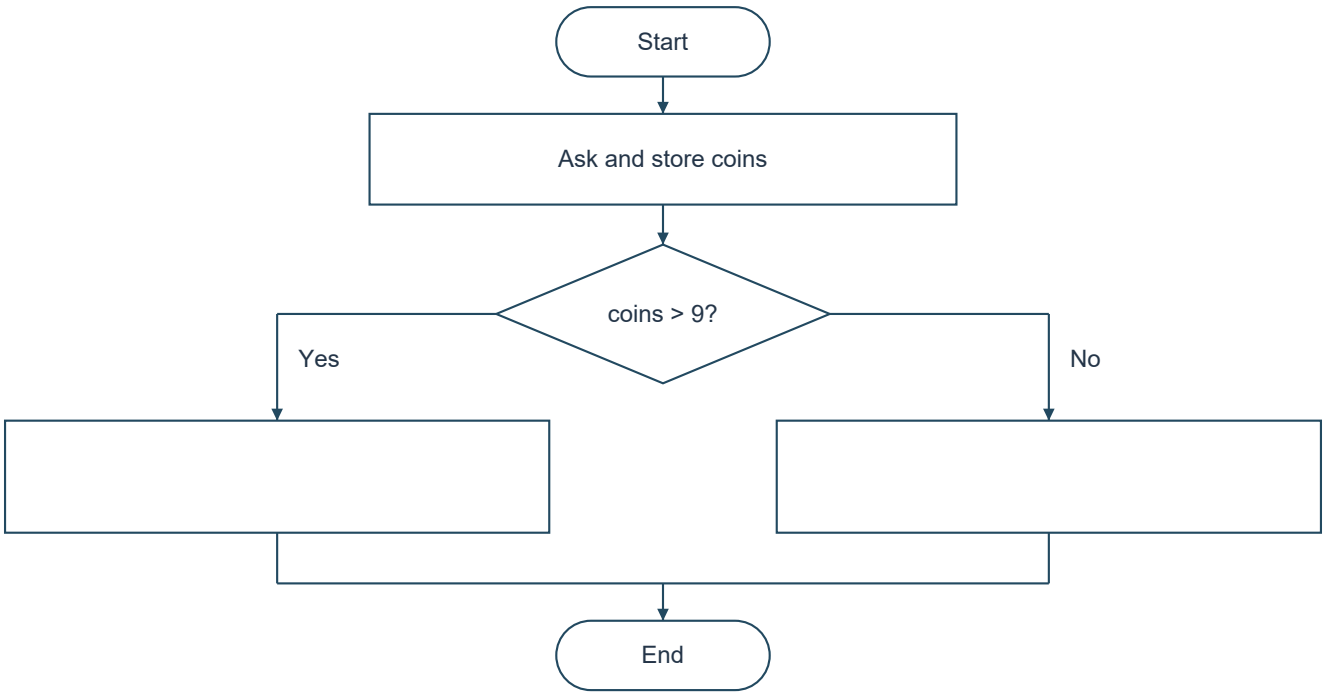
Predict the message. Then run the program and record what actually happens.

1 Click the flag for **each** test. Enter the number in the first column. Keep the two messages exactly as they are on page 2.

Coins	Expected message	Actual message
9	Collect more coins.	
10	Gate open!	
11		
0		
25		

Show the decision

Complete the two empty boxes. Use the two messages from your Scratch code.



Choose a different whole number and predict before running the code.

Coins: _____ Expected: _____

Actual: _____

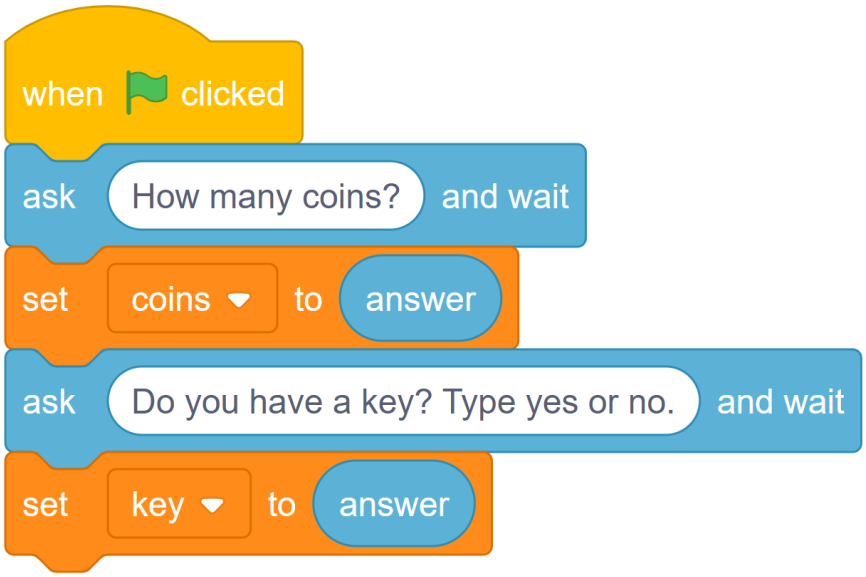
Remember whether there is a key

Add a second question. Keep the coins answer and the key answer separate.

---

1. Keep your working gate. In **Variables**, choose **Make a Variable**. Name it **key**, choose **For all sprites**, and click **OK**. Tick key so you can see its value.
2. From **Sensing**, add a second **ask ... and wait** block. Place it directly below **set coins to answer**, before the if/else block. Type the new question shown below.
3. From **Variables**, add **set ... to ...** directly below the new question. Choose **key** from its menu. Drag a new light-blue **answer** block into its value input.

The top of your script now looks like this



Keep your if/else block attached below this stack. We will change its condition on page 5. For now, it still checks the coins only.

Check the two stored answers

Click the flag. Enter **9**, then enter **yes** for the key question. Look at the variable values on the Stage.

Variable	Should show	What did yours show?
coins	9	
key	yes	

Important: answer holds the most recent reply. Store the coins reply before asking about the key. If coins shows yes, change the second set block to set key.

Require coins AND a key

The gate opens only when both conditions are true.

Find the blocks: Operators (green): and, > and =. Variables (orange): coins and key.

1. Drag your coins comparison out of **if**. Keep it in the Code area; you will use it in the new test.



2. Make a second comparison with **=**. Put the orange oval **key** on the left. Type **yes** on the right.



3. Drag out **and** from Operators. Put the coins comparison in the left hole and the key comparison in the right hole.



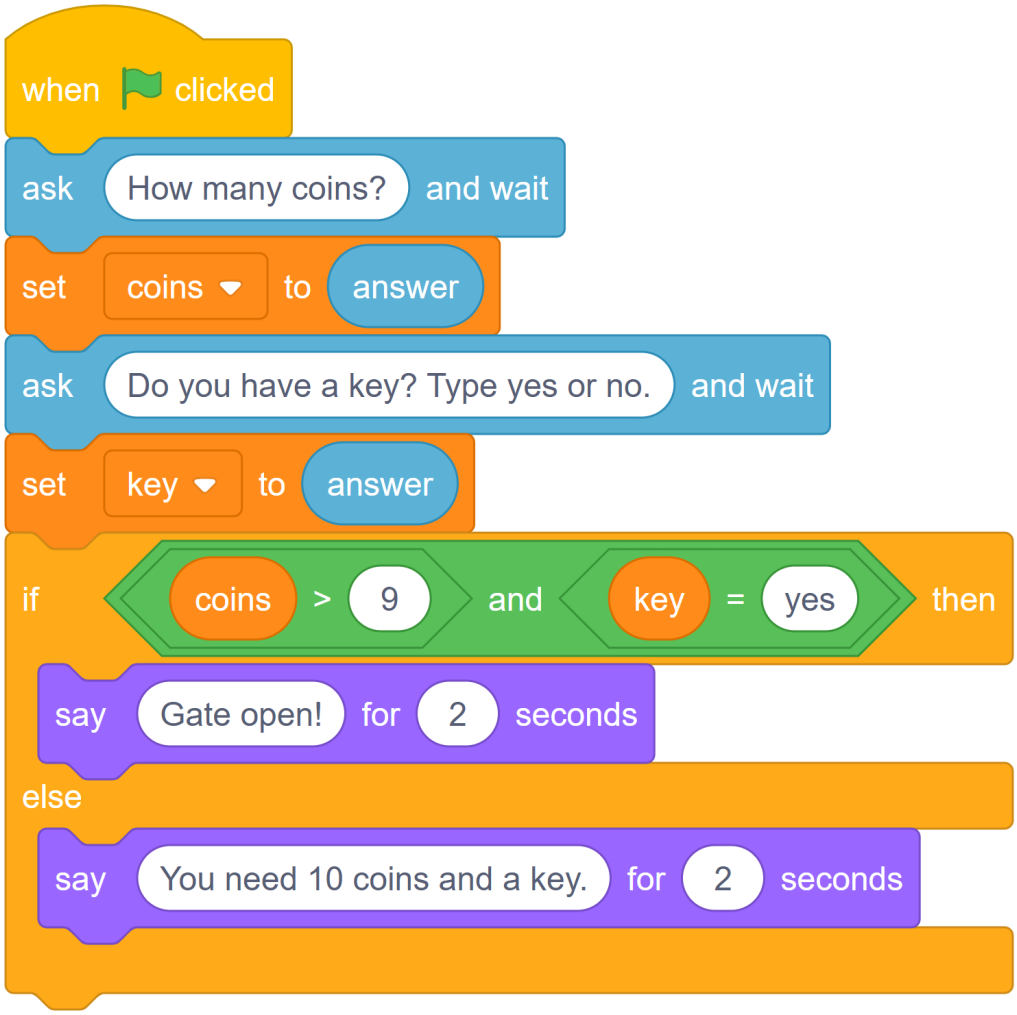
4. Put the whole combined green test into the empty hexagonal hole in **if**.

5. Keep **Gate open!** above else. Below else, change the message to **You need 10 coins and a key.**

Read your rule: There must be at least 10 coins AND the key answer must be yes. Check the complete script on the next page.

Check the complete AND gate

Compare your script with this picture. Start at the green-flag block.



Test: Open means Gate open! Closed means the else message.

Coins	Key	Expected	Actual
9	yes	Closed	
10	no	Closed	
10	yes	Open	
11	yes		

If a test fails: Check that the green block says and, the second set block says key, and both purple messages are inside if/else. Save your working project.

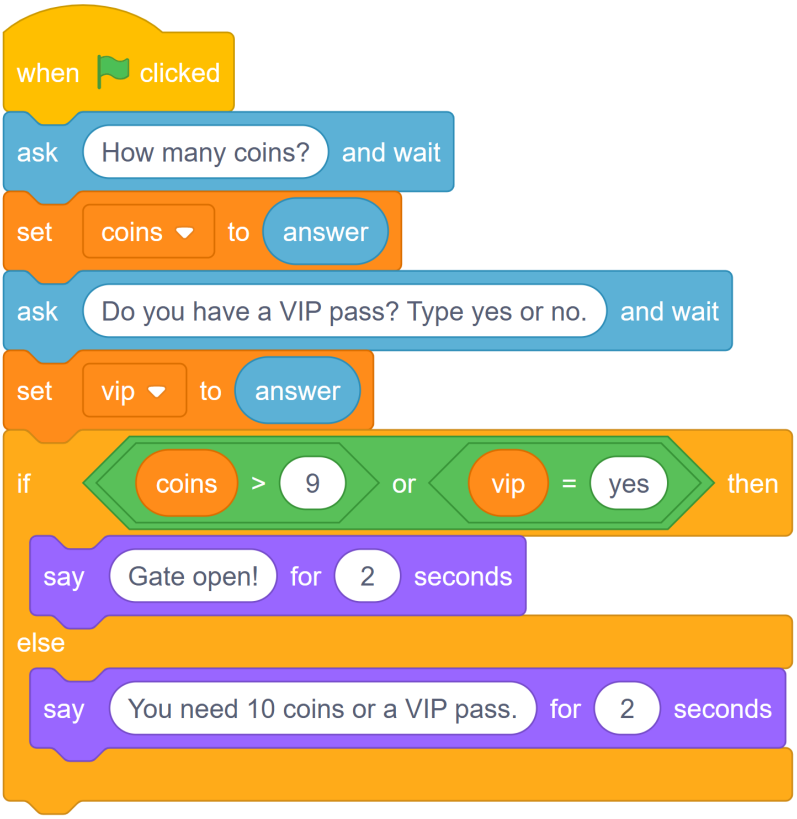
EXTENSION - OR

Try coins OR a VIP pass

Save a copy first. This is a different gate rule.

---

1. Choose **File > Save to your computer**. Save a copy as **Bonus_Gate.sb3**. Keep your original coin-and-key file.
2. Make **vip** (For all sprites). Change the second question as shown. In the second set block, choose **vip** instead of key.
3. Replace **and** with **or**. Keep coins > 9 on the left. On the right use **vip = yes**, not key = yes. Change the else message to match the script.



OR: One true condition is enough; both true also opens the gate. The old key variable is unused.

Predict Open or Closed, then test each row.

Coins	VIP pass	Expected	Actual
9	no	Closed	
9	yes		
10	no		
10	yes		

OPTIONAL - NOT

NOT means the opposite

Paper task: predict the result. You do not need to change your Scratch gate.

A warning should sound when the temperature is **above 30°C** and the alert is **not muted**. Here, muted = yes means the sound is switched off.

Read the NOT block first

Find **not** in **Operators** (green). It reverses a true/false result.



muted	muted = yes	not (muted = yes)
yes	True	False
no	False	True

Now read the whole condition



Read it as: **temperature is above 30 AND the alert is NOT muted.** Both tests must be true for the warning to sound.

Predict: does the warning sound?

Temperature	Muted?	Warning: yes or no?
30°C	no	
31°C	no	
31°C	yes	

Why does exactly 30°C not trigger the warning?

Search and sort

Paper task: use number cards or write the numbers.

Linear search: look from left to right

7	2	9	4	1	6

1. Find **4**. Start with 7. Check one card at a time and stop as soon as you find 4. Count every card you check, including 4.

   Number of comparisons: _____

2. Put the cards back in the same order. Search for **8**. Keep checking until you find it or reach the end.

   Number of comparisons: _____ Found or missing? _____

Insertion sort: build a sorted group

Start with 7. Take the next card and insert it into the correct place in the sorted group. Use each card once.

Next card	Sorted group after inserting it
Start	7
2	2, 7
9	2, 7, 9
4	
1	
6	

Explain

The original cards are not in numerical order. Why must you check every card before deciding that 8 is missing?